

Note technique 11 - SMTP_SERVICE

Gmail OAuth2 dans G2M

1. Objet

Cette note technique décrit le processus de mise en place du support d'envoi d'emails transactionnels dans G2M avec Gmail SMTP et une authentification OAuth2.

L'objectif est de permettre à G2M d'envoyer des emails applicatifs, notamment :

- liens d'activation de compte ;
- emails de test depuis le panneau d'administration ;
- futures notifications transactionnelles ;
- messages de supervision ou d'alerte.

Le principe retenu est de ne pas stocker le mot de passe personnel Gmail dans l'application. G2M utilise à la place une authentification OAuth2 auprès de Google, puis Nodemailer se charge de présenter un jeton OAuth2 au serveur SMTP Gmail.

2. Contexte G2M

Le projet G2M utilise actuellement :

Zone	Élément
Backend	Node.js / Express
Vues	EJS
Base de données	SQLite avec <code>better-sqlite3</code>
Authentification applicative	JWT en cookie <code>HttpOnly</code>
Emails	<code>nodemailer</code> via <code>services/mailService.js</code>
Paramètres persistants	table <code>settings</code>
Administration	panneau <code>/admin</code> protégé par <code>requireAuth</code> et <code>requireRole("admin")</code>

Avant cette note, le service mail supportait déjà :

- un mode développement, lorsque la configuration SMTP est absente ;
- un mode SMTP classique avec `SMTP_USER` et `SMTP_PASSWORD`.

Le besoin additionnel consiste à ajouter un troisième cas d'usage : Gmail SMTP avec OAuth2.

3. Rappel conceptuel

3.1. SMTP

SMTP signifie Simple Mail Transfer Protocol. C'est le protocole utilisé pour envoyer des emails.

Dans le cas de G2M, l'application ne remet pas directement les emails aux destinataires finaux. Elle se connecte à un serveur SMTP sortant, par exemple `smtp.gmail.com`, et lui demande d'envoyer le message.

Le serveur SMTP exige généralement :

- un hôte : `SMTP_HOST` ;
- un port : `SMTP_PORT` ;
- une indication de chiffrement : `SMTP_SECURE` ;
- un compte expéditeur ou utilisateur : `SMTP_USER` ;
- une preuve d'autorisation.

Dans une configuration SMTP classique, la preuve d'autorisation est un mot de passe SMTP ou un mot de passe d'application.

3.2. Email transactionnel

Un email transactionnel est un email envoyé automatiquement par une application à la suite d'un événement précis.

Exemples dans G2M :

- un administrateur invite un utilisateur ;
- un utilisateur demande un lien d'activation ;
- un test SMTP est lancé depuis `/admin/email-test` ;
- une alerte future est déclenchée par une anomalie.

Un email transactionnel n'est pas une newsletter. Il doit être fiable, traçable et envoyé uniquement à des destinataires concernés.

3.3. OAuth2

OAuth2 est un mécanisme d'autorisation. Dans ce contexte, OAuth2 permet à G2M d'obtenir le droit d'envoyer des emails via un compte Gmail sans connaître ni stocker le mot de passe personnel du compte.

Le flux repose sur trois éléments importants :

Élément	Rôle
<code>client_id</code>	Identifie l'application OAuth2 créée dans Google Cloud Console
<code>client_secret</code>	Secret associé au client OAuth2
<code>refresh_token</code>	Jeton long terme permettant d'obtenir des jetons d'accès courts

Nodemailer utilise ces informations pour générer automatiquement un jeton d'accès OAuth2, puis s'authentifie auprès du serveur SMTP Gmail via le mécanisme XOAUTH2.

3.4. Différence entre mot de passe d'application et OAuth2

Option	Principe	Avantage	Limite
Mot de passe d'application Gmail	Google génère un mot de passe dédié	Simple à configurer	Reste un secret statique proche d'un mot de passe
OAuth2 Gmail	G2M utilise un client OAuth2 et un refresh token	Plus conforme aux pratiques modernes	Mise en place plus technique

Pour G2M, OAuth2 est retenu comme approche plus propre conceptuellement, surtout pour éviter l'exposition du mot de passe personnel Gmail.

4. Architecture retenue dans G2M

4.1. Service concerné

Le fichier principal est :

```
services/mailService.js
```

Ce service :

- lit la configuration email ;
- détecte le mode d'authentification ;
- construit le transport Nodemailer ;
- envoie le message ;
- bascule en mode développement si la configuration est incomplète.

4.2. Paramètres persistants

Les paramètres SMTP sont stockés dans la table SQLite `settings`, créée dans :

```
config/database.js
```

Les nouveaux paramètres OAuth2 sont :

Clé settings	Type	Description
smtp.auth_method	string	password ou oauth2
gmail.oauth_client_id	secret	Client ID OAuth2 Google
gmail.oauth_client_secret	secret	Client secret OAuth2 Google
gmail.oauth_refresh_token	secret	Refresh token OAuth2 Google

Les valeurs de type `secret` sont masquées dans l'interface d'administration. Si un champ secret est soumis vide, l'ancienne valeur est conservée.

4.3. Variables d'environnement

Le fichier `.env.example` contient désormais :

```
MAIL_FROM=no-reply@g2m.local SMTP_AUTH_METHOD=password SMTP_HOST= SMTP_PORT=
SMTP_SECURE=false SMTP_USER= SMTP_PASSWORD= GMAIL_OAUTH_CLIENT_ID=
GMAIL_OAUTH_CLIENT_SECRET= GMAIL_OAUTH_REFRESH_TOKEN=
```

Pour Gmail OAuth2, la configuration cible est :

```
MAIL_FROM=operagis2022@gmail.com SMTP_AUTH_METHOD=oauth2 SMTP_HOST=smtp.gmail.com
SMTP_PORT=465 SMTP_SECURE=true SMTP_USER=operagis2022@gmail.com SMTP_PASSWORD=
GMAIL_OAUTH_CLIENT_ID=... GMAIL_OAUTH_CLIENT_SECRET=... GMAIL_OAUTH_REFRESH_TOKEN=...
```

Alternative avec STARTTLS :

```
SMTP_HOST=smtp.gmail.com SMTP_PORT=587 SMTP_SECURE=false
```

Les deux modes peuvent fonctionner. Le port 465 avec `SMTP_SECURE=true` est explicite et simple à lire. Le port 587 avec `SMTP_SECURE=false` utilise STARTTLS.

5. Implémentation technique

5.1. Détection de configuration complète

Le service vérifie d'abord que la base SMTP est présente :

- `MAIL_FROM` ;
- `SMTP_HOST` ;
- `SMTP_PORT`.

Ensuite, il vérifie l'authentification selon le mode :

Mode	Champs requis
password	<code>SMTP_USER</code> , <code>SMTP_PASSWORD</code>
oauth2	<code>SMTP_USER</code> , <code>GMAIL_OAUTH_CLIENT_ID</code> , <code>GMAIL_OAUTH_CLIENT_SECRET</code> , <code>GMAIL_OAUTH_REFRESH_TOKEN</code>

Si la configuration n'est pas complète, G2M n'envoie pas réellement l'email. Il écrit le message en mode développement dans les logs.

5.2. Authentification SMTP classique

Pour un fournisseur comme Brevo, SMTP2GO ou Mailjet, Nodemailer reçoit :

```
auth: {   user: env.SMTP_USER,   pass: env.SMTP_PASSWORD }
```

5.3. Authentification Gmail OAuth2

Pour Gmail OAuth2, Nodemailer reçoit :

```
auth: {   type: "OAuth2",   user: env.SMTP_USER,   clientId: env.GMAIL_OAUTH_CLIENT_ID,   clientSecret: env.GMAIL_OAUTH_CLIENT_SECRET,   refreshToken: env.GMAIL_OAUTH_REFRESH_TOKEN }
```

Nodemailer se charge ensuite d'obtenir un jeton d'accès OAuth2 valide et de présenter l'authentification au serveur SMTP Gmail.

5.4. Masquage dans l'administration

La page :

```
/admin/email-test
```

affiche l'état SMTP sans révéler les secrets.

Exemple attendu :

```
MAIL_FROM : operagis2022@gmail.com SMTP_AUTH_METHOD : oauth2 SMTP_HOST :  
smtp.gmail.com SMTP_PORT : 465 SMTP_USER : operagis2022@gmail.com SMTP_PASSWORD : Non  
renseigné GMAIL_CLIENT_ID : ***** GMAIL_CLIENT_SECRET : *****  
GMAIL_REFRESH_TOKEN : *****
```

Si tous les éléments requis sont présents, l'état passe à :

```
prêt
```

6. Processus Google Cloud

6.1. Créer un projet Google Cloud

1. Ouvrir Google Cloud Console.

2. Créer un projet, par exemple `G2M SMTP Service`.
3. Vérifier que le compte Gmail utilisé est celui qui doit envoyer les emails.

6.2. Configurer l'écran de consentement OAuth

4. Ouvrir `APIs & Services`.
5. Aller dans `OAuth consent screen`.
6. Choisir le type adapté.
7. Renseigner le nom de l'application, par exemple `G2M`.
8. Ajouter l'adresse email de support.
9. Ajouter l'adresse Gmail concernée dans les utilisateurs de test si l'application reste en mode test.

Pour un usage interne avec un seul compte Gmail, le mode test peut suffire. Pour un déploiement plus large, Google peut exiger une vérification.

6.3. Créer un client OAuth2

10. Aller dans `Credentials`.
11. Créer un identifiant `OAuth client ID`.
12. Choisir un type de client adapté.
13. Récupérer :
 - o `client_id`;
 - o `client_secret`.

Ces deux valeurs correspondent à :

```
GMAIL_OAUTH_CLIENT_ID=... GMAIL_OAUTH_CLIENT_SECRET=...
```

6.4. Obtenir un refresh token

Le refresh token doit être obtenu après consentement du compte Gmail expéditeur.

Le scope requis pour SMTP Gmail est :

```
https://mail.google.com/
```

Ce scope est sensible, car il donne un accès large à la messagerie. Il faut donc limiter l'usage à un compte dédié à G2M, par exemple :

```
no-reply-g2m@gmail.com
```

ou, pour une première phase :

```
operagis2022@gmail.com
```

L'obtention du refresh token peut se faire avec un petit script OAuth2 local, Google OAuth Playground ou un outil interne. L'important est que le jeton obtenu soit ensuite stocké dans G2M comme secret et jamais exposé dans les logs.

7. Configuration G2M

7.1. Configuration via le panneau admin

Dans G2M :

14. Se connecter avec un compte `admin`.

15. Ouvrir :

```
/admin/settings
```

16. Dans la section email/SMTP, renseigner :

```
mail.from = operagis2022@gmail.com smtp.auth_method = oauth2 smtp.host =  
smtp.gmail.com smtp.port = 465 smtp.secure = true smtp.user = operagis2022@gmail.com  
gmail.oauth_client_id = ... gmail.oauth_client_secret = ... gmail.oauth_refresh_token  
= ...
```

17. Enregistrer.

18. Ouvrir :

```
/admin/email-test
```

19. Vérifier que l'état SMTP est `prêt`.

20. Envoyer un email de test.

7.2. Configuration via Render

En production Render, il est préférable de stocker les secrets dans les variables d'environnement.

Variables recommandées :

```
MAIL_FROM=operagis2022@gmail.com SMTP_AUTH_METHOD=oauth2 SMTP_HOST=smtp.gmail.com  
SMTP_PORT=465 SMTP_SECURE=true SMTP_USER=operagis2022@gmail.com SMTP_PASSWORD=  
GMAIL_OAUTH_CLIENT_ID=... GMAIL_OAUTH_CLIENT_SECRET=... GMAIL_OAUTH_REFRESH_TOKEN=...
```

Après modification des variables Render :

21. redémarrer le service ;
22. ouvrir `/admin/email-test` ;
23. vérifier l'état ;
24. envoyer un email de test.

8. Sécurité

8.1. Ne pas utiliser le mot de passe personnel Gmail

Le mot de passe personnel Gmail ne doit jamais être stocké dans :

- `.env` ;
- SQLite ;
- Render ;
- logs ;
- tickets ;
- documentation ;
- captures d'écran.

OAuth2 remplace cette exposition par des secrets techniques révoquables.

8.2. Utiliser un compte dédié

Il est préférable de ne pas utiliser une adresse personnelle pour l'envoi transactionnel.

Compte recommandé :

```
no-reply-g2m@gmail.com
```

ou, avec un domaine professionnel :

```
no-reply@domaine-g2m.org
```

L'adresse `MAIL_FROM` doit correspondre à l'utilisateur Gmail autorisé. Gmail peut réécrire ou refuser certains expéditeurs non autorisés.

8.3. Protéger les secrets

Les secrets OAuth2 doivent être traités comme des mots de passe :

- masqués dans l'interface ;
- exclus des logs ;
- exclus de Git ;
- stockés dans Render comme variables secrètes en production ;
- renouvelés si une fuite est suspectée.

8.4. Révocation

En cas de compromission :

25. révoquer l'accès OAuth dans le compte Google ;
26. supprimer ou renouveler le client secret ;
27. générer un nouveau refresh token ;
28. mettre à jour G2M ;
29. tester l'envoi ;
30. contrôler les derniers logs d'audit.

8.5. Limites Gmail

Gmail n'est pas toujours le meilleur fournisseur transactionnel pour une application en production. Il impose des limites et des contrôles anti-abus.

Pour une phase pilote, Gmail OAuth2 est acceptable. Pour un usage massif ou institutionnel, un fournisseur transactionnel comme Brevo, SMTP2GO, Mailjet, SendGrid, Mailgun ou Amazon SES peut devenir plus adapté.

9. Tests

9.1. Tests automatisés ajoutés

La suite de tests G2M couvre désormais :

- accès au panneau admin ;
- protection des routes admin ;

- persistance des settings ;
- masquage des secrets ;
- rapport SQLite ;
- email test en mode développement ;
- reconnaissance d'une configuration Gmail OAuth2 complète.

Commande utilisée :

```
npm.cmd test
```

Résultat obtenu :

```
50 tests 50 pass 0 fail
```

9.2. Test manuel recommandé

31. Configurer OAuth2 Gmail.
32. Ouvrir `/admin/email-test`.
33. Vérifier que l'état est `prêt`.
34. Envoyer un email vers une adresse de test.
35. Vérifier la réception.
36. Créer une invitation utilisateur.
37. Tenter une connexion avec l'email invité.
38. Vérifier que le lien d'activation est envoyé.
39. Activer le compte.
40. Vérifier la connexion.

10. Diagnostic des erreurs fréquentes

10.1. État SMTP incomplet

Cause probable :

- `SMTP_HOST` vide ;
- `SMTP_PORT` vide ;
- `SMTP_USER` vide ;
- `smtp.auth_method = oauth2`, mais un secret OAuth2 manque.

Correction :

- compléter les paramètres dans `/admin/settings` ou Render ;
- redémarrer l'application si les variables Render ont été modifiées ;
- vérifier `/admin/email-test`.

10.2. Authentification refusée

Causes possibles :

- refresh token invalide ;
- mauvais client secret ;
- scope OAuth2 incomplet ;
- compte Gmail non autorisé dans l'écran de consentement ;
- compte différent entre `SMTP_USER` et le compte qui a consenti.

Correction :

- régénérer le refresh token ;
- vérifier le scope `https://mail.google.com/` ;
- vérifier que `SMTP_USER` correspond au compte Gmail autorisé.

10.3. Email non reçu

Causes possibles :

- email classé en spam ;
- Gmail a limité l'envoi ;
- `MAIL_FROM` incohérent ;
- destinataire incorrect ;
- contenu bloqué.

Correction :

- tester vers une autre adresse ;
- vérifier les spams ;
- utiliser un sujet simple ;
- vérifier les logs G2M ;
- envisager un fournisseur transactionnel dédié si le volume augmente.

11. Fichiers impactés

Fichier	Rôle
<code>services/mailService.js</code>	Ajout du mode OAuth2 et construction de l'auth Nodemailer
<code>config/database.js</code>	Ajout des settings OAuth2 par défaut
<code>models/Setting.js</code>	Ajout des clés éditables OAuth2
<code>views/admin/email-test.ejs</code>	Affichage sécurisé de l'état OAuth2
<code>.env.example</code>	Documentation des variables OAuth2
<code>tests/app.test.js</code>	Test de configuration Gmail OAuth2 complète

12. Sources techniques

Documentation officielle et références utiles :

- Google - XOAUTH2 protocol for Gmail IMAP, POP and SMTP :
<https://developers.google.com/workspace/gmail/imap/xoauth2-protocol>
<https://developers.google.com/workspace/gmail/imap/xoauth2-protocol>

- Google - OAuth 2.0 for Web Server Applications :
<https://developers.google.com/identity/protocols/oauth2/web-server>
(<https://developers.google.com/identity/protocols/oauth2/web-server>)
- Nodemailer - OAuth2 authentication : <https://nodemailer.com/smtp/oauth2>
(<https://nodemailer.com/smtp/oauth2>)
- Nodemailer - Using Gmail : <https://nodemailer.com/usage/using-gmail>
(<https://nodemailer.com/usage/using-gmail>)

13. Conclusion

Le support Gmail SMTP OAuth2 permet à G2M d'envoyer des emails transactionnels sans stocker le mot de passe personnel Gmail. L'application reste compatible avec le SMTP classique tout en ajoutant un mode plus conforme aux pratiques modernes de sécurité.

La mise en place demande une étape externe dans Google Cloud Console pour obtenir `client_id`, `client_secret` et `refresh_token`. Une fois ces valeurs configurées dans G2M ou dans Render, le panneau `/admin/email-test` permet de vérifier rapidement que le service SMTP est prêt.

Pour un pilote ou un faible volume, Gmail OAuth2 est une solution acceptable. Pour une exploitation institutionnelle à plus grand volume, il faudra évaluer un fournisseur transactionnel dédié avec domaine vérifié, SPF, DKIM et DMARC.